

Design Rationale REQ3: The People of the Valley

1. Introduction

This document outlines the design rationale for Requirement 3: "The People of the Valley." This requirement introduces three distinct Non-Playable Characters (NPCs): Sellen, Merchant Kale, and Guts, each with unique characteristics, behaviors, and conditional monologue systems, into the "ELDEN THING: THE VALLEY OF THE INHERITREE" game. The primary goal of this design is to seamlessly integrate these NPCs and their interactive dialogue mechanics into the existing game framework. This rationale will adhere to the classes, interfaces, and relationships depicted in the provided UML diagram for REQ3, while also considering object-oriented principles (SOLID) to ensure clarity, maintainability, and extensibility for future additions of NPCs or more complex interaction scenarios.

2. Design Goals

The key objectives for implementing REQ3 are:

1. NPC Implementation:

- Create NpcSellen (s) with 150 HP.
- Create NpcKale (k) with 200 HP.
- Create NpcGuts (g) with 500 HP.
- Enable all NPCs to wander the map using a shared WanderBehaviour.

2. Monologue System Implementation:

- Allow the Farmer (Player) to interact with NPCs to listen to their monologues using a ListenAction.
- Implement a monologue pool for each NPC, from which a monologue is randomly selected upon interaction.
- Support conditional monologues for Kale (based on Farmer's runes, inventory, and proximity to cursed entities) and Guts (based on Farmer's health) using a flexible Condition system.

3. NPC-Specific Behavior Implementation:

- Implement Guts's aggressive behavior: he will attack any actor with more than 50 HP using his bare fists (25 damage, 50% hit rate), managed by an AttackBehaviour.

4. Extensibility and Reusability:

- Design the system to easily accommodate new NPCs with unique or existing conditional monologues.
- Allow for the creation of new conditions for monologues or other NPC behaviors.
- Ensure that the condition-checking mechanisms are reusable.

5. Adherence to OOP Principles:

- Apply SOLID principles (Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion) to create a robust, maintainable, and scalable design.

3. Design Analysis & Decisions

This section breaks down Requirement 3 into key functional areas, analyzing design alternatives for each, with direct reference to the REQ3 UML diagram.

3.1 Feature: NPC Core Structure and Basic Interaction

This feature addresses how NPCs (Sellen, Kale, Guts) are fundamentally represented in the game and how the Player (Farmer) initiates interaction with them for dialogue.

- Chosen Design (Option A - Aligns with REQ3 UML):

- Description:
 - An abstract class `game.actors.npc.Npc` extends `engine.actors.Actor`. This class serves as the base for all specific NPCs, providing common functionalities such as managing a collection of Behaviours (e.g., a default `WanderBehaviour`).
 - Concrete NPC classes (`game.actors.npc.NpcSellen`, `game.actors.npc.NpcKale`, `game.actors.npc.NpcGuts`) inherit from `Npc`. Each subclass is responsible for its specific attributes (HP) and implementing its unique set of monologues.
 - Interaction for dialogue is initiated by the Farmer. When the Farmer is near an NPC, the NPC's `allowableActions()` method (inherited or overridden from `Npc`) will include a `game.actors.npc.ListenAction`. This `ListenAction` targets the specific NPC instance.
- Maintainability: Good. Common NPC logic is centralized in the `Npc` base class, reducing redundancy. Specific NPC characteristics are encapsulated within their respective subclasses. Adding new NPCs would involve creating a new subclass of `Npc`.
- Pros & Cons:
 - Pros:
 - Clear Type Hierarchy (LSP): Establishes `Npc` as a clear subtype of `Actor`, allowing NPCs to be treated polymorphically.
 - Code Reusability (DRY): Common behaviors like wandering and the mechanism for offering `ListenAction` are in the `Npc` class.
 - Encapsulation: Each NPC subclass encapsulates its specific data (HP, monologue definitions).
 - SRP (for `ListenAction`): The `ListenAction` class has the single responsibility of handling the listening interaction.
 - Direct UML Alignment: This structure directly reflects the `Npc` abstract class, its concrete subclasses, and the `ListenAction` as depicted in the REQ3 UML diagram.
 - Cons:
 - If future NPCs require vastly different fundamental interaction mechanisms not suited to `ListenAction`, the base `Npc` might need modification or a different abstraction.
- Alternative Design (Option B): Individual NPC Classes Extending `Actor` Directly, with Dialogue Logic in `Player`
 - Description: Each NPC (`Sellen`, `Kale`, `Guts`) would directly extend `engine.actors.Actor`. There would be no common `Npc` base class. The `Player` class would contain logic to check if a nearby actor is an instance of a specific NPC type (e.g., `instanceof NpcSellen`). If so, the `Player` would then generate a specific listening action or directly call a dialogue method on that concrete NPC.

- Maintainability: Poor. Adding new NPCs or changing interaction logic would require modifications to the Player class, violating OCP. Common NPC behaviors like wandering would need to be duplicated or managed by a separate utility.
- Pros & Cons:
 - Pros:
 - Might appear superficially simpler if there are very few, very distinct NPCs.
 - Cons:
 - High Code Duplication (Violates DRY): Common NPC setup or interaction checks would be repeated in the Player class or duplicated across NPC classes.
 - Poor Extensibility (Violates OCP): Adding a new NPC type would necessitate changes to the Player class to recognize and interact with it.
 - Tight Coupling: The Player class would become tightly coupled to all concrete NPC types.
 - Violation of LSP Principle: Treating all NPCs uniformly would be difficult without a common base type, leading to extensive instanceof checks.
- Decision & Justification (Option A):

Option A is selected as it directly aligns with the provided REQ3 UML diagram and strongly adheres to object-oriented principles. The abstract Npc class provides a solid foundation for common NPC functionalities and ensures that all NPCs can be treated polymorphically (e.g., when scanning for nearby NPCs to interact with). This structure promotes code reuse (DRY) for behaviors like wandering and the general mechanism of offering a ListenAction. Encapsulating NPC-specific details within their respective subclasses (LSP, SRP) makes the system more organized and easier to maintain and extend compared to the tightly coupled and less scalable Option B.

3.2 Feature: Monologue System and Conditional Dialogue Delivery

This feature details how NPCs store, manage, and deliver their lines of dialogue, especially those that depend on specific game conditions.

- Chosen Design (Option A - Aligns with REQ3 UML):
 - Description:
 - A `game.actors.npc.Monologue` class is defined, responsible for holding a single string of dialogue text. Crucially, each Monologue instance is associated with an instance of a `game.conditions.Condition` interface.
 - The `game.conditions.Condition` interface defines a contract (e.g., a `check()` method, possibly taking context like Actor listener, GameMap map) for evaluating if the monologue should be available.
 - Concrete condition classes (e.g., `game.conditions.LowHealthCondition`, `game.conditions.LowRunesCondition`,

game.conditions.EmptyInventoryCondition, game.conditions.NearbyCapabilityCondition, and game.conditions.DefaultCondition) implement the Condition interface. These are instantiated with appropriate parameters (e.g., the Farmer actor for LowRunesCondition, the NPC's location and the CURSED capability for NearbyCapabilityCondition). The UML diagram shows these condition classes and their relationship with Monologue and other relevant entities like Wallet and GeneralCapability.

- Each concrete NPC class (e.g., NpcKale) overrides a method like getMonologues(Actor listener, GameMap map) from the Npc base. This method dynamically constructs a list of Monologue objects, instantiating the correct Condition for each.
- When the ListenAction is executed, it calls this getMonologues() method on the target NPC. It then iterates through the returned Monologues, calls check() on each associated Condition. Only monologues whose conditions are met are added to a temporary pool. A random monologue is then selected from this pool for display.
- Maintainability: Excellent. New dialogue lines can be added by creating new Monologue objects. New triggering logic can be added by creating new Condition implementations without altering the core NPC or ListenAction logic.

- Pros & Cons:

Pros:

- High Cohesion (SRP): The Monologue class is responsible for the dialogue text and its trigger condition. Each Condition class handles a specific check.
- Excellent Extensibility (OCP): New conditions and monologues can be added by creating new classes/instances without modifying the core Npc or ListenAction.
- Decoupling (DIP): The Npc and ListenAction classes depend on the Condition abstraction rather than concrete checking logic.
- Reusability: Condition classes (e.g., LowHealthCondition) can be parameterized and reused for different NPCs or even other game mechanics.
- Clear UML Alignment: This design accurately reflects the Monologue, Condition interface, and various concrete condition classes shown in the REQ3 UML.

- Cons:

- May lead to a larger number of small Condition classes if conditions become very granular and are not designed for reusability with parameters.
- Alternative Design (Option B): Hardcoded Conditional Logic in NPC Methods
 - Description: Instead of separate Condition objects, the logic for determining

monologue availability (e.g., if (farmer.getRunes() < 500)) would be directly embedded within each NPC's getMonologues() method using if-else if statements. The ListenAction would simply call this method and pick from the pre-filtered list.

- Maintainability: Poor. As the number of conditions or NPCs grows, these methods would become very long, complex, and difficult to manage (violating SRP). Changes to a condition shared by multiple NPCs would require editing multiple classes.

- Pros & Cons:

Pros:

- Might seem initially simpler for a very small number of NPCs with very few, simple conditions.

- Cons:

- Violates OCP: Adding new conditions or changing logic for existing ones requires modifying the NPC class methods.
- Low Cohesion: NPC classes become responsible for both their core identity and complex conditional dialogue logic.
- Poor Reusability: Conditional logic is tightly coupled to specific NPC methods and not easily reused.
- Difficult to Test: Large methods with complex conditional blocks are harder to unit test effectively.

- Decision & Justification (Option A):

Option A is chosen due to its superior modularity, extensibility, and adherence to OOP principles, particularly SRP and OCP, as well as its direct correspondence to the REQ3 UML diagram. The Strategy Pattern, as implemented by the Condition interface and its concrete classes, allows for clean separation of condition-checking logic from the Monologue data and the NPC classes themselves. This makes the system easy to update with new dialogues and new triggering rules without cascading changes, leading to a much more maintainable and robust design in the long run.

3.3 Feature: Guts's Aggressive Behavior

This feature covers how Guts's specific combat AI (attacking actors with >50 HP) is implemented.

- Chosen Design (Option A - Aligns with REQ3 UML):

- Description:

- A game.behaviours.AttackBehaviour class implements the engine.actors.Behaviour interface.
- An instance of this AttackBehaviour is added to NpcGuts's behaviours map (likely with a higher priority than its WanderBehaviour to ensure attacking is considered first).

- The `AttackBehaviour.getAction(Actor actor, GameMap map)` method:
 1. Scans the surroundings of actor (Guts).
 2. For each adjacent Actor (target), it checks if the target's HP (obtained via `target.getAttribute(BaseActorAttributes.HEALTH)`) is greater than 50.
 3. If such a target is found, it returns a new `game.weapons.actions.AttackAction(target, direction, actor.getIntrinsicWeapon())`. As per REQ3, Guts uses his bare fists. The REQ3 UML diagram shows `NpcGuts` associated with `AttackBehaviour` and `BareFist`.
 - Maintainability: Good. Guts's specific attack logic is encapsulated within the `AttackBehaviour`. If his targeting criteria change, modifications are localized to this behavior class.
 - Pros & Cons:
 - Pros:
 - SRP: The `AttackBehaviour` is solely responsible for the logic of deciding when and whom Guts should attack.
 - Modularity: Separates combat AI from other NPC concerns (like wandering or dialogue).
 - Clear AI Structure: Fits naturally into the engine's existing behaviour-based system for NPCs.
 - Direct UML Alignment: This design matches the `AttackBehaviour` associated with `NpcGuts` in the REQ3 UML diagram, which also shows `NpcGuts` having a `BareFist` intrinsic weapon.
 - Cons:
 - The current `AttackBehaviour` (as described for Guts) is specific to the ">50 HP" rule. If other NPCs need different attack conditions, this class might need generalization or new variants.
- Alternative Design (Option B): `Attack Logic Directly in NpcGuts.playTurn()`
 - Description: All logic for scanning surroundings, checking target HP, and creating/returning an `AttackAction` would be embedded directly within the `NpcGuts.playTurn()` method. This logic would likely be checked before or after its `WanderBehaviour`.
 - Maintainability: Poor. The `playTurn()` method would become overly complex, mixing movement AI with combat AI, thus violating SRP and making the class harder to understand and modify.
 - Pros & Cons:
 - Pros:

- Might seem slightly more straightforward for a single NPC with a very simple, fixed attack rule.
- Cons:
 - Violates SRP: The playTurn() method would be responsible for too many distinct concerns (overall turn orchestration, movement AI, combat AI).
 - Low Cohesion: Attack logic would be intertwined with other unrelated AI logic within the same method.
 - Poor Reusability: The specific attack logic would be difficult to reuse for other NPCs without copy-pasting, leading to maintenance issues.
 - Harder to Test: Large, complex playTurn() methods with multiple responsibilities are more challenging to unit test thoroughly.
- Decision & Justification (Option A):

Option A is the preferred design as it adheres to the behaviour-driven architecture of the game engine and the specifics of the REQ3 UML diagram. Encapsulating Guts's aggressive logic within a dedicated AttackBehaviour promotes SRP, making the NpcGuts class cleaner and its playTurn() method simpler. This modularity also makes the attack behavior easier to understand, test, and potentially modify in isolation. For future extensibility, as noted in the design considerations, this AttackBehaviour could be made more generic (e.g., by accepting a Condition object to define attack eligibility) or specialized through inheritance if vastly different attack patterns are needed for other NPCs.

4. Addressing Design Considerations from REQ3

- Other NPCs with new conditional monologues:
 - The chosen design (Abstract Npc, Monologue class, Condition interface) is highly extensible. New NPCs can be created by subclassing Npc. New conditional monologues can be added by creating new Monologue instances associated with new or existing Condition implementations.
- NPCs using existing conditions:
 - Existing Condition classes (e.g., LowHealthCondition, NearbyCapabilityCondition as shown in the REQ3 UML) can be parameterized (e.g., LowHealthCondition could take a threshold; NearbyCapabilityCondition could take the specific capability like GeneralCapability.CURSED to check for) and reused across different NPCs for their monologues.
- Other conditions for attacking (e.g., Guts only attacks if target HP > 50 HP):
 - The current AttackBehaviour for Guts encapsulates this specific ">50 HP" condition. To support more varied attack conditions for future NPCs:
 1. The AttackBehaviour could be made more generic by accepting a Condition object in its constructor. This Condition would then be checked to determine if an attack should proceed against a potential target. (This aligns well with the Condition pattern used for monologues).
 2. Alternatively, different specialized AttackBehaviour subclasses could be created for distinct attack patterns/conditions. The current design is sufficient for REQ3's specific requirement for Guts, but option 1 (parameterizing with a Condition) offers better long-term flexibility without class proliferation for minor variations in attack triggers.

5. Overall Design Summary for Requirement 3

The design for "REQ3: The People of the Valley" establishes a robust framework for NPCs within the "ELDEN THING" game. It centers around an abstract `Npc` class, from which specific characters like `NpcSellen`, `NpcKale`, and `NpcGuts` are derived. Player interaction for dialogue is facilitated by a `ListenAction`, which interfaces with the NPCs' monologue systems. The monologue system itself is made dynamic and extensible through `Monologue` objects, each associated with a `Condition` (implementing the `Condition` interface and realized by concrete classes like `LowHealthCondition`, `LowRunesCondition`, etc., as per the REQ3 UML diagram). This allows for context-sensitive dialogue. Standard NPC movement relies on a reusable `WanderBehaviour`. Guts's unique aggression is encapsulated within a dedicated `AttackBehaviour`, which checks the target's HP before initiating an `AttackAction` with his `BareFist`. This component-based and strategy-pattern-influenced design promotes SOLID principles, particularly SRP and OCP, ensuring the system is maintainable and can be readily extended with new NPCs, dialogue, conditions, and behaviors in future development cycles.

6. Conclusion

The proposed design for Requirement 3 effectively introduces interactive NPCs with distinct personalities and conditional dialogues into the game world. By utilizing an abstract base class for NPCs, a data-driven `Monologue` system linked to a flexible `Condition` strategy pattern, and specific `Behaviour` classes for actions such as attacking and wandering, the system successfully meets all specified functionalities. The design directly reflects the structure and relationships indicated in the REQ3 UML diagram. This approach not only fulfills the immediate requirements but also establishes a maintainable and extensible foundation, addressing the key design considerations for future enhancements involving new characters, more intricate dialogue triggers, and diverse NPC behaviors, all while adhering to sound object-oriented design principles.